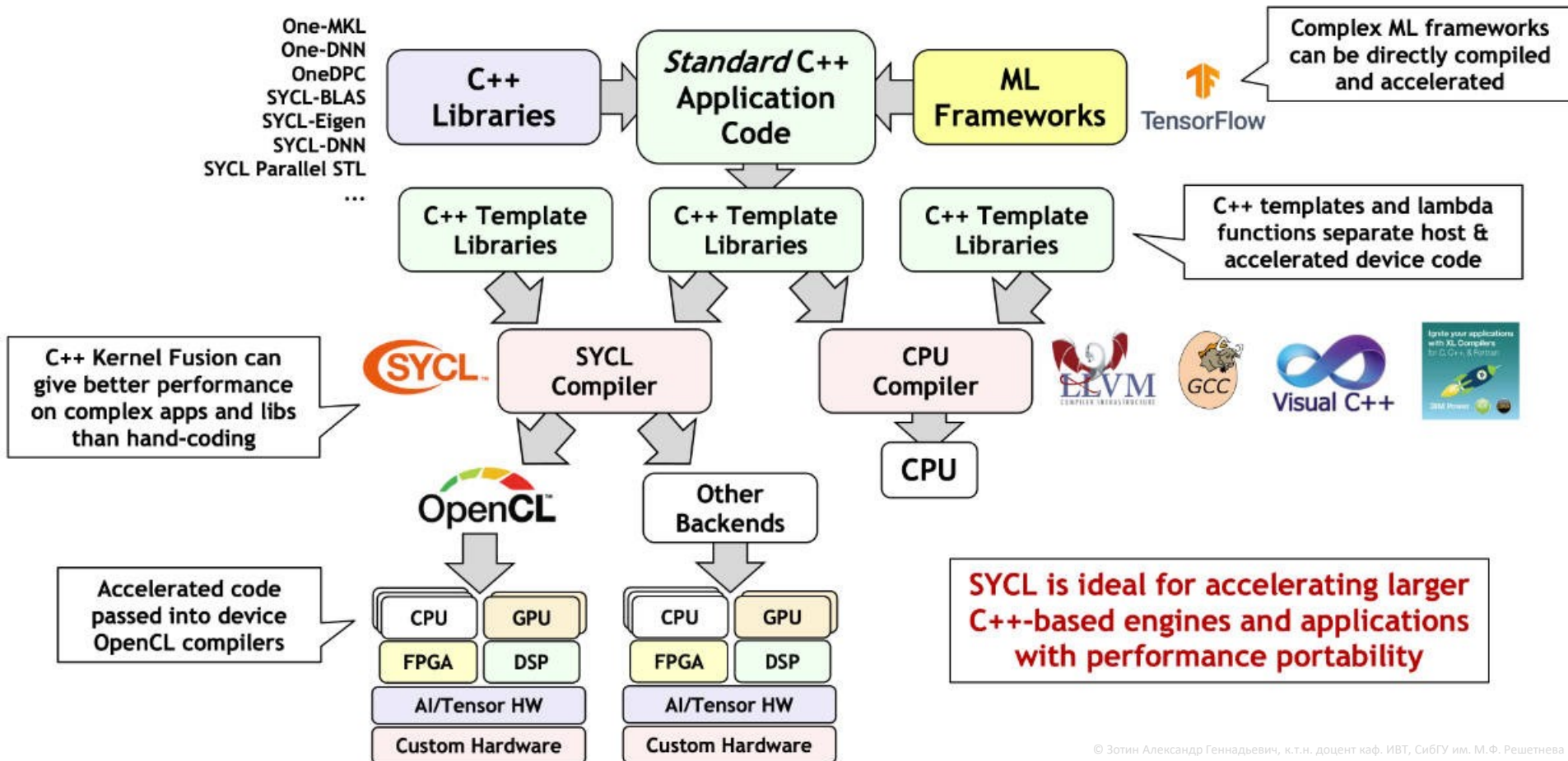


Знакомство с Intel[®] oneAPI Data Parallel C++ (DPC++)

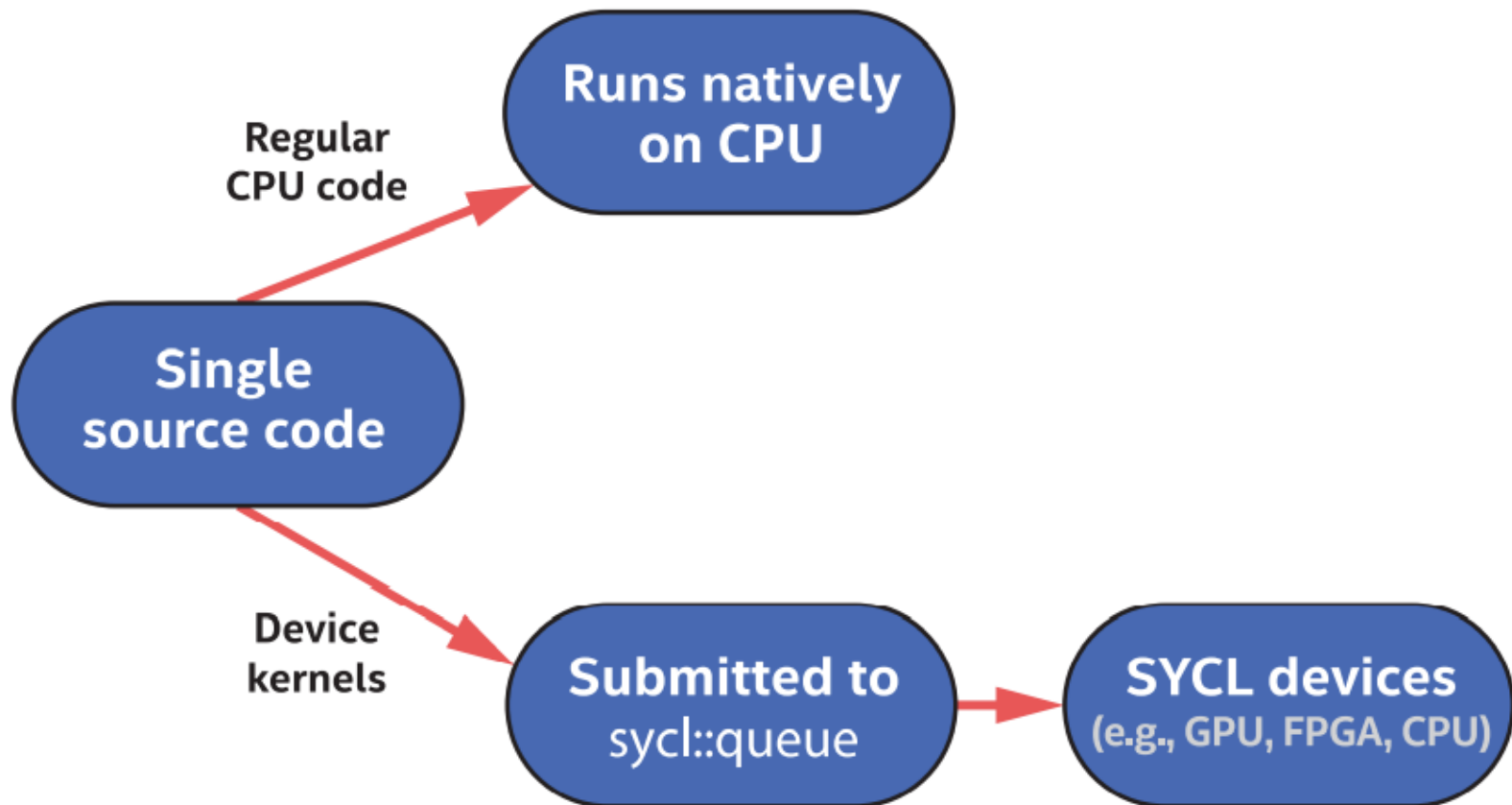
SYCL

SYCL - это модель программирования более высокого уровня для повышения производительности программирования на различных аппаратных ускорителях. SYCL - это бесплатный кроссплатформенный уровень абстракции, основанный на базовых концепциях, переносимости и эффективности, вдохновленных OpenCL, который позволяет писать код для гетерогенных процессоров в стиле «единого источника» с использованием полностью стандартного C++.



Исполнение кода

Программы SYCL могут быть с одним исходником, что означает, что одна и та же единица трансляции (обычно исходный файл и его заголовки) содержит как код, определяющий ядра вычислений (kernels), которые должны выполняться на устройствах SYCL, так и код хоста, который управляет выполнением этих ядер.



Data Parallel C++

DPC++ = ISO C++, Khronos SYCL и расширения общности (community extensions)

Свобода выбора: модель программирования, готовая к будущему

- Позволяет повторно использовать код на разных аппаратных компонентах
- Настройка под специфический ускоритель
- Открытая межотраслевая альтернатива проприетарному языку

DPC++ = ISO C++ and Khronos SYCL and community extensions

- Повышение продуктивности C++ за счет применения общих, знакомых конструкций C и C++
- Добавление SYCL от Khronos Group для параллелизма данных и гетерогенного программирования.

Direct Programming:
Data Parallel C++

Community Extensions

Khronos SYCL

ISO C++

Intel® oneAPI DPC++/C++ Compiler

Производительность и эффективность параллельного программирования

Компилятор для обеспечения бескомпромиссной производительности и производительности параллельного программирования на процессорах и ускорителях

- Позволяет повторно использовать код для разных аппаратных компонентов, разрешая при этом подстройку для конкретного ускорителя.
- Открытая межотраслевая альтернатива проприетарному языку с единой архитектурой

Примеры кода (Code samples):

tinyurl.com/dpcpp-tests

tinyurl.com/oneapi-samples

oneAPI DPC++/C++ Compiler and Runtime

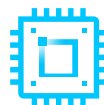
DPC++ Source Code

Clang/LLVM

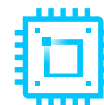
<https://github.com/intel/llvm>

DPC++ Runtime

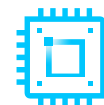
<https://github.com/intel/compute-runtime>



CPU



GPU



FPGA

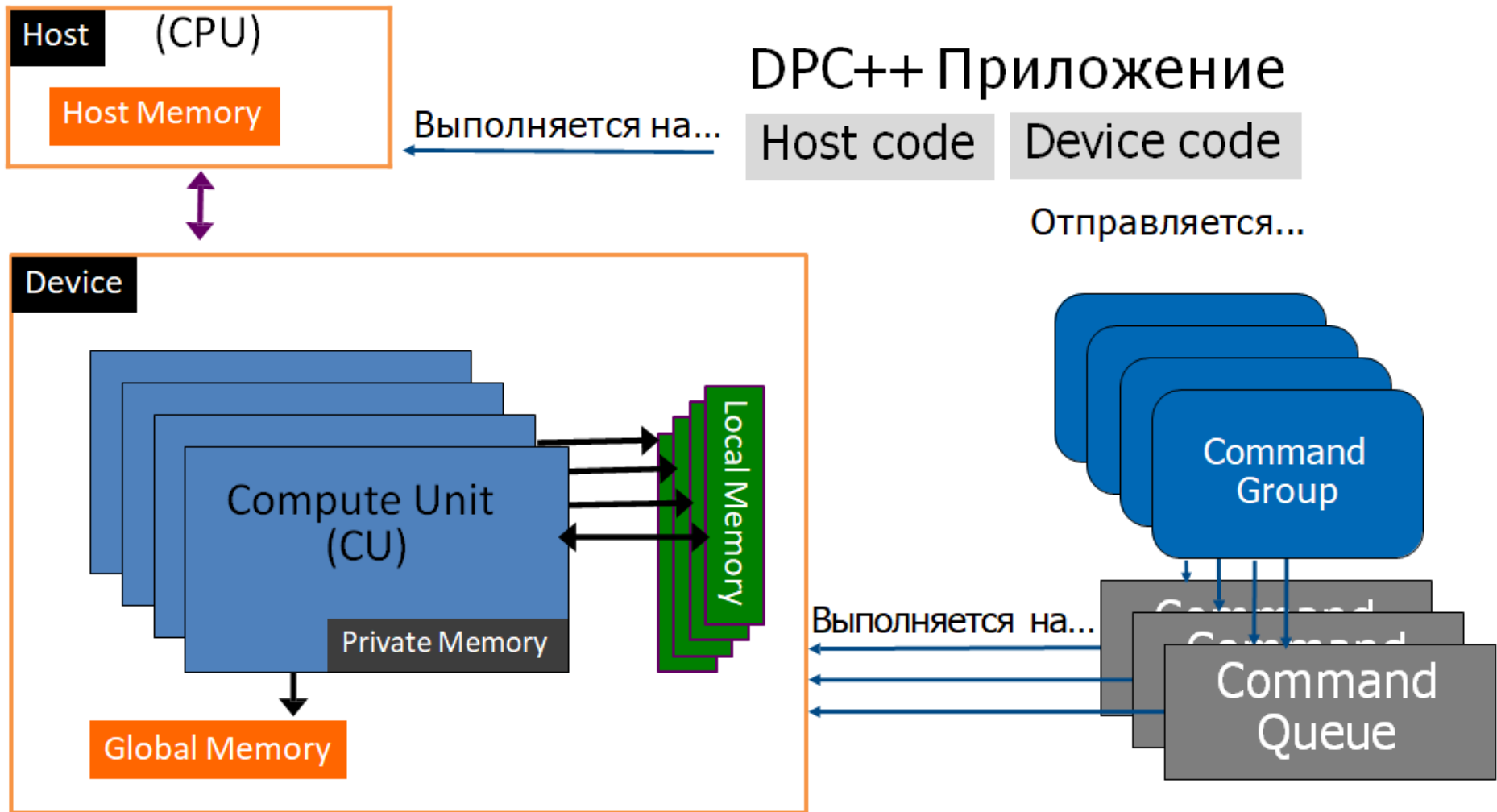
Intel® oneAPI DPC++/C++ Compiler

Intel Compiler	Target	Поддержка OpenMP	OpenMP Offload Support	Included in oneAPI Toolkit
Intel® C++ Compiler, IL0 icc	CPU	Да	Нет	HPC
Intel® oneAPI DPC++/C++ Compiler dpcpp	CPU, GPU, FPGA*	Да	Да	Base
Intel® oneAPI DPC++/C++ Compiler icx	CPU GPU*	Да	Да	Base
Intel® Fortran Compiler, IL0 ifort	CPU	Да	Нет	HPC
Intel® Fortran Compiler Beta ifx	CPU, GPU*	Да	Да	HPC

Codeplay DPC++ Compiler для Nvidia GPUs

- Разработчики могут изменять сборку и повторно использовать код между как для ускорителей NVIDIA так и Intel из единой исходной базы (исходников).
- Codeplay - первый в отрасли разработчик oneAPI, реализовавший инструментарий разработчика на основе спецификаций oneAPI.
- Они использовали проект с открытым исходным кодом на основе DPC ++ LLVM, разработанный Intel.
- Codeplay - это одна из ключевых компаний продвигающая стандарт SYCL Khronos, на основе DPC ++
- Более подробная информация а [блоге](#)
- DPC++ toolchain
с поддержкой NVIDIA CUDA: tinyurl.com/dpcpp-cuda-be
tinyurl.com/dpcpp-cuda-be-webinar

Основы DPC++



Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

Host code

Код основной
программы

**Accelerator
device code**

Host code

Асинхронное выполнение

Host

**Исполнение
Host code**

Добавление
ядра (kernel)
в graph,
(очередь)

и
продолжение
исполнения

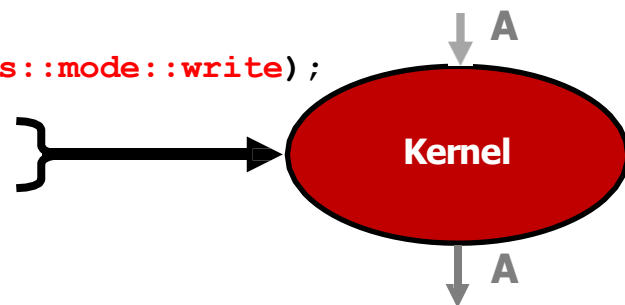
```
#include <CL/sycl.hpp>
using namespace sycl;
```

```
int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q(gpu_selector{});
        q.submit([&](handler &cgh) {
            auto A = bufA.get_access(cgh, access::mode::read);
            auto B = bufB.get_access(cgh, access::mode::read);

            auto C = bufC.get_access(cgh, access::mode::write);
            cgh.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    for (int i = 0; i < 1024; i++)
        std::cout << C[i] << std::endl;
}
```

Graph

Graph
выполняется
асинхронно от
host программы



Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;
```

```
int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
```

```
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
```

```
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

Область
приложения

Область
группы команд

Область
устройства

Область
приложения

Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

Создание буферов на основе host векторов/указателей

Буферы инкапсулируют данные в приложении SYCL

- На двух устройствах (host и ускорителе)

Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

Очередь (queue)
отправляет группы
команд для
выполнения средой
исполнения SYCL.

queue - это
механизм, с
помощью которого
работа отправляется
на устройство.

Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

**Создание аксессоров
(Accessors creation)**

**Механизм доступа к
данным буфера**

**Создает
зависимости
данных в графе
SYCL, которые
упорядочивают
выполнение ядра.**

Структура DPC++ Приложения

```
#include <CL/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // Некоторая инициализация
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for<class vector_add> (1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i]
                    << std::endl;
}
```

Ядро (Kernel)
сложения векторов
ставит задачу
`parallel_for` в очередь.

Передаёт объект
функции / лямбда
выражения, который
будет выполняться
каждым рабочим
элементом

```
h.parallel_for<class vector_add> (1024, [=](auto i) {
    C[i] = A[i] + B[i];
});
```

Граф исполнения ядер (Kernel)

```
int main() {  
    auto R = range<1>{ num };  
    buffer<int> A{ R }, B{ R };  
    queue Q;
```

```
    Q.submit([&](handler& h) {  
        accessor out(A, h, write_only);  
        h.parallel_for(R, [=](id<1> idx) {  
            out[idx] = idx[0]; }); });
```

➔ **Kernel 1**

```
    Q.submit([&](handler& h) {  
        accessor out(A, h, write_only);  
        h.parallel_for(R, [=](id<1> idx) {  
            out[idx] = idx[0]; }); });
```

➔ **Kernel 2**

```
    Q.submit([&](handler& h) {  
        accessor out(B, h, write_only);  
        h.parallel_for(R, [=](id<1> idx) {  
            out[idx] = idx[0]; }); });
```

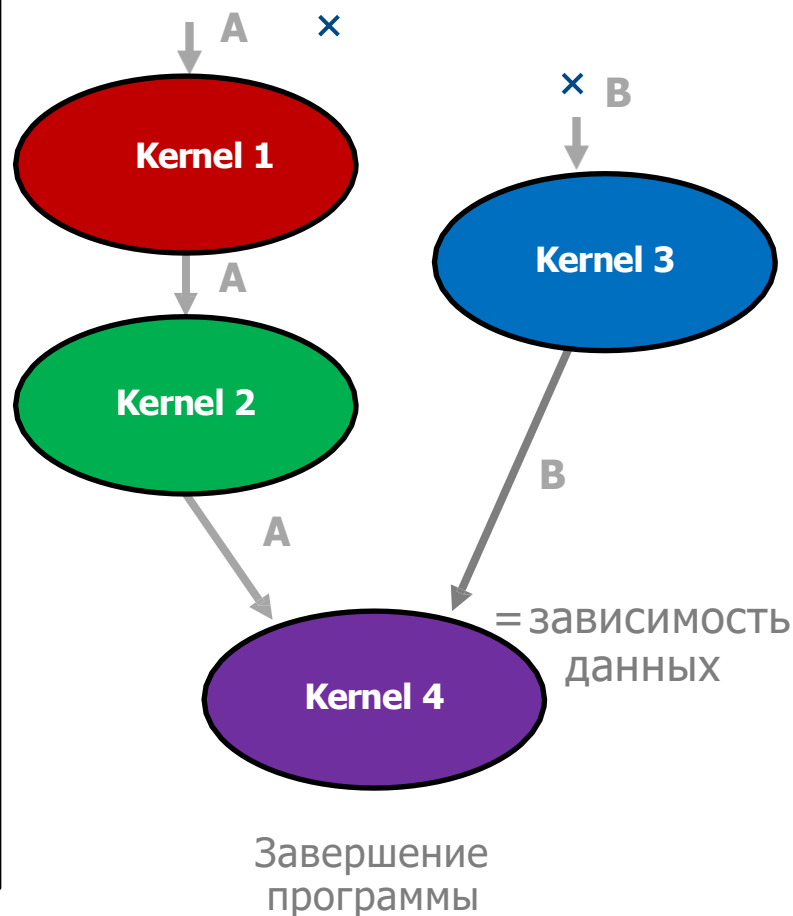
➔ **Kernel 3**

```
    Q.submit([&](handler& h) {  
        accessor in(A, h, read_only);  
        accessor inout(B, h);  
        h.parallel_for(R, [=](id<1> idx) {  
            inout[idx] *= in[idx]; }); });
```

➔ **Kernel 4**

```
}
```

Автоматическое разрешение
зависимости данных и
управления!



Функции DPC ++ для вызова ядер

```
h.single_task(
```

```
[=] () {  
    // функция ядра выполняется ТОЧНО один раз для ОДНОГО рабочего  
    // элемента  
});
```

```
h.parallel_for(
```

```
    range<3>(1024,1024,1024), // используем 3D в этом примере  
    [=](id<3> myID) {  
        // функция ядра выполняется в n-мерном диапазоне (NDrange)  
    });
```

```
h.parallel_for(
```

```
    nd_<3>({1024,1024,1024},{16,16,16}), // используем 3D в этом примере  
    [=](nd_rangeitem<3> myID) {  
        // функция ядра выполняется в n-мерном диапазоне (NDrange)  
    });
```

```
h.parallel_for_work_group(
```

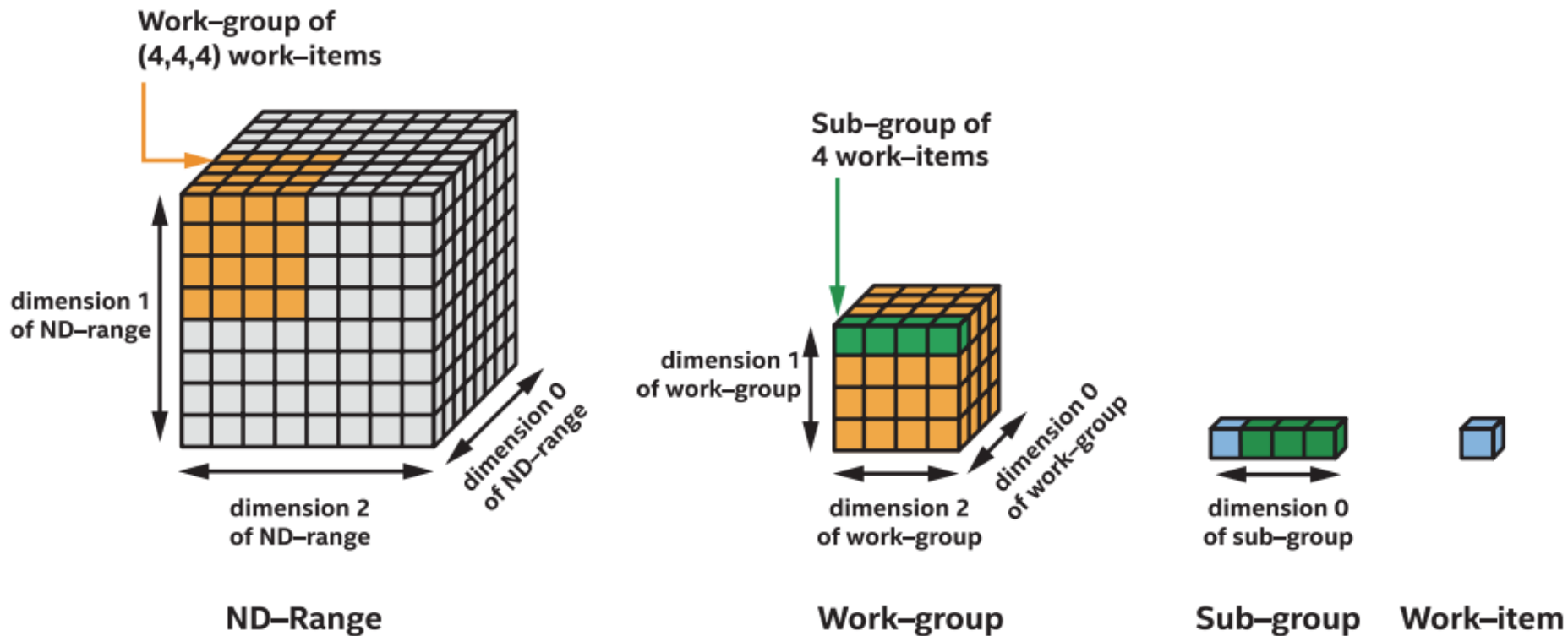
```
    range<2>(1024,1024), // используем 2D в этом примере  
    [=](group<2> grp) {  
        // функция ядра выполняется один раз для каждой рабочей группы  
    });
```

```
grp.parallel_for_work_item(
```

```
    range<1>(1024), // используем 1D в этом примере  
    // функция ядра выполняется один раз для каждого рабочего элемента  
    [=](h_item<1> myItem) {  
    });
```

Представление 3D ND-range

Трёхмерный ND-диапазон (ND-range) делится на рабочие группы (work-groups), подгруппы(sub-groups) и рабочие элементы (work-items).



Основы параллельных ядер (Kernels)

Функциональность базовых параллельных ядер представлена через классы диапазона (**range**), идентификатора (**id**) и элемента (**item**).

- ❑ класс **range** используется для описания итерационного пространства параллельного выполнения
- ❑ класс **id** используется для индексации отдельного экземпляра ядра при параллельном выполнении
- ❑ класс **item** представляет собой отдельный экземпляр функции ядра, предоставляет дополнительные функции для запроса свойств диапазона выполнения

```
h.parallel_for(range<1>(1024), [=](id<1> idx){  
    // Код для исполнения на устройстве  
});
```

```
h.parallel_for(range<1>(1024), [=](item<1> item){  
    auto idx = item.get_id();  
    auto R = item.get_range();  
    // Код для исполнения на устройстве  
});
```

Синхронизация

Синхронизация внутри функции ядра

- ☐ Барьеры для синхронизации рабочих элементов в рабочей группе
- ☐ Нет примитивов синхронизации между рабочими группами

Синхронизация между хостом и устройством

- ☐ Вызов метода `wait ()` очереди устройства (queue)
- ☐ Уничтожение буфера синхронизирует данные с памятью хоста
- ☐ Конструктор средств доступа хоста - это заблокированный вызов, который возвращается только после того, как все ядра в очереди, работающие с этим буфером завершают выполнение.
- ☐ Построение DAG (Directed Acyclic Graph) из функциональных объектов группы команд, поставленных в очередь устройства

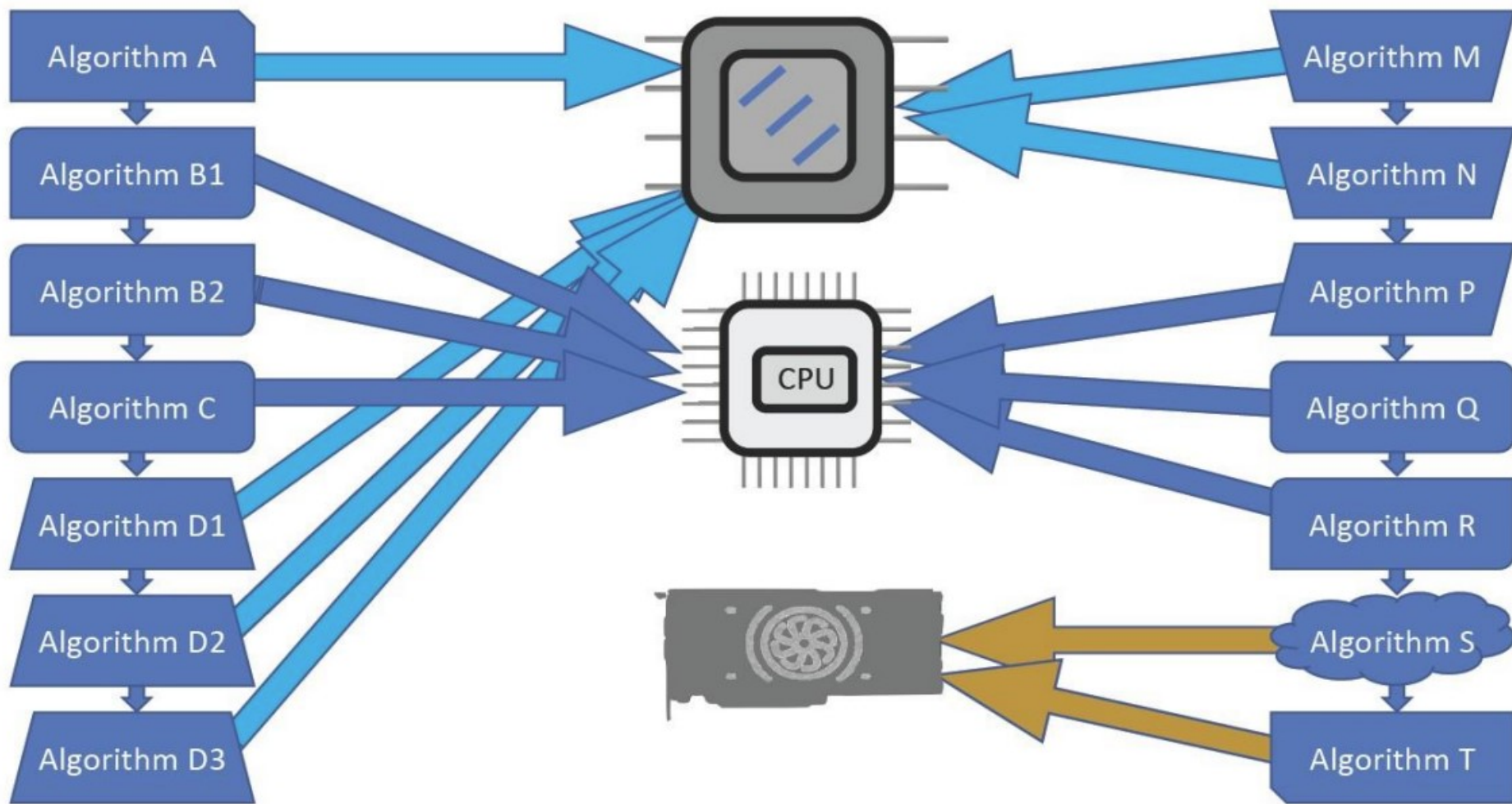
Место исполнения кода (где выполнять код ядра)

Определение устройства (любое устройство):	<code>queue q (); // default_selector</code>
Создание очереди, ориентированной на предварительно настроенные классы устройств:	<code>queue q(cpu_selector{}); queue q(gpu_selector{}); queue q(intel::fpga_selector{}); queue q(accelerator_selector{}); queue q(host_selector{});</code> SYCL 1.2.1
Создание очереди для специфичного устройства (настраиваемые критерии):	<code>class custom_selector : public device_selector { int operator()(..... // Any logic you want! ... queue q(custom_selector{});</code>

default_selector

- Среда выполнения DPC ++ оценивает все устройства и выбирает одно с максимальной вычислительной мощностью
- Переменная окружения

Место исполнения кода (где выполнять код ядра)



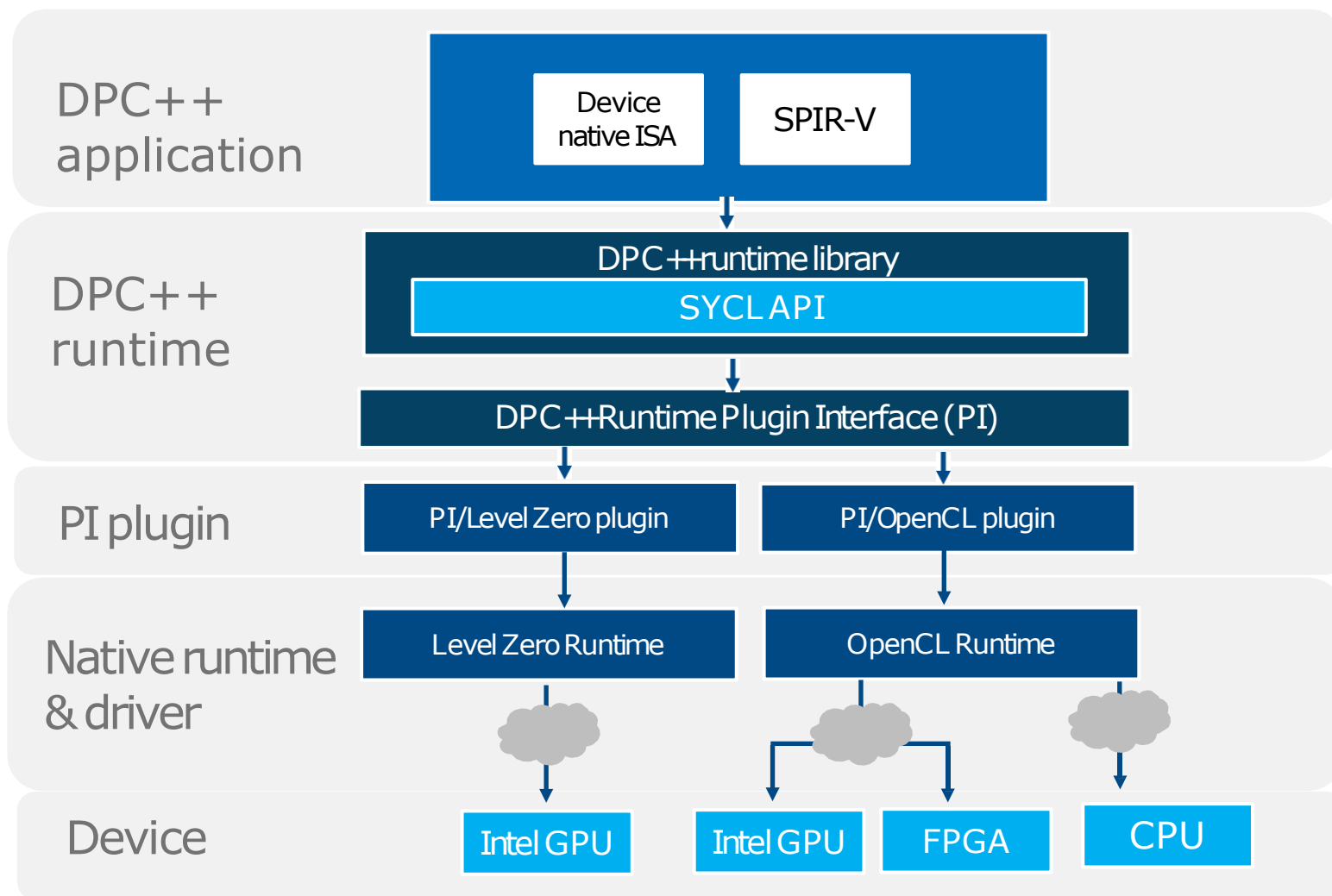
Пример: Просмотр сведений о доступных устройствах

```
#include <CL/sycl.hpp>
using namespace sycl;
...
void show_platforms() {
    auto platforms = platform::get_platforms();

    for (auto& platform : platforms) {
        std::cout << "Platform: "
        << platform.get_info<info::platform::name>()
        << std::endl;

        auto devices = platform.get_devices();
        for (auto& device : devices) {
            std::cout << "  Device: "
            << device.get_info<info::device::name>()
            << std::endl;
        }
    }
}
```

Архитектура исполнения



Управляется через переменную окружения SYCL_BE : PI_OPENCL, PI_LEVEL0

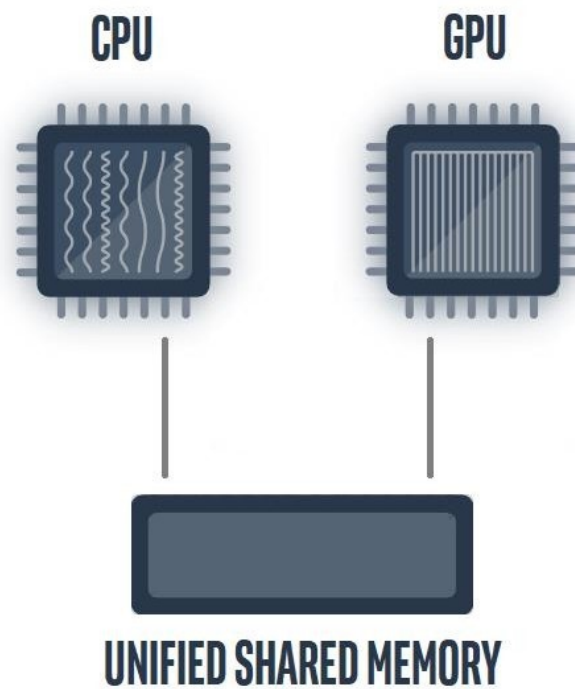
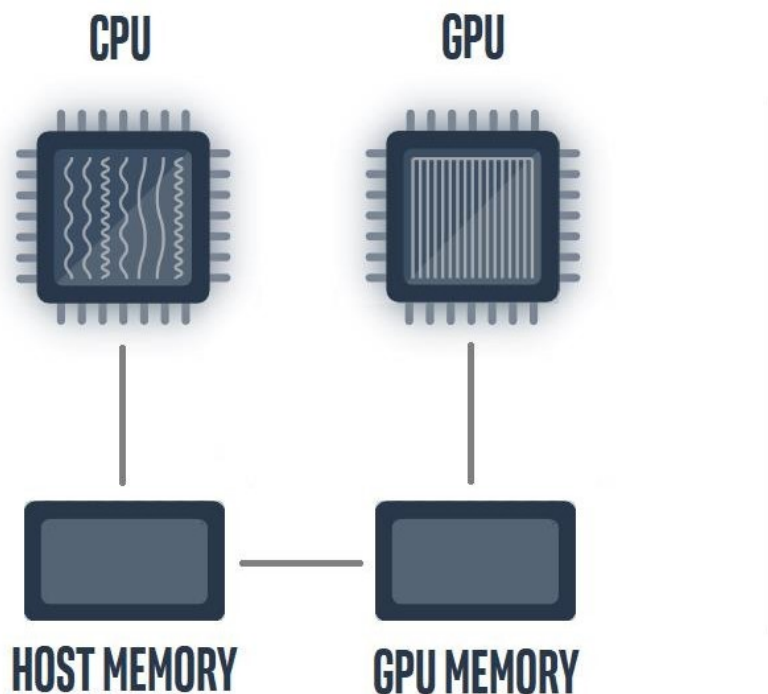
больше информации : tinyurl.com/dpcpp-pi, tinyurl.com/levelzero-spec

Unified Shared Memory

Unified Shared Memory (USM) предоставляет альтернативу на основе указателей в DPC ++.

- Упрощает перенос на ускоритель.
- Предоставляет программистам желаемый уровень контроля.
- Дополняет буферы

Разработчики могут ссылаться на один и тот же объект памяти в коде хоста и устройства с помощью USM.



Интерпретация USM

```
#include <CL/sycl.hpp>
using namespace sycl;
```

```
int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // some data initialization
    {
        buffer bufA {A}, bufB {B}, bufC {C};
```

```
        queue q(gpu_selector{});
        q.submit([&](handler &cgh) {
            auto A = bufA.get_access(cgh, access::mode::read);
            auto B = bufB.get_access(cgh, access::mode::read);
            auto C = bufC.get_access(cgh, access::mode::write);
            cgh.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
```

```
    }
    for (int i = 0; i < 1024; i++) {
        std::cout << C[i] << std::endl;
    }
}
```

```
#include <CL/sycl.hpp>
using namespace sycl;
```

```
int main() {
```

```
    queue q(gpu_selector{});
    int *A = malloc_shared<int>(1024, q);
    int *B = malloc_shared<int>(1024, q);
    int *C = malloc_shared<int>(1024, q);
```

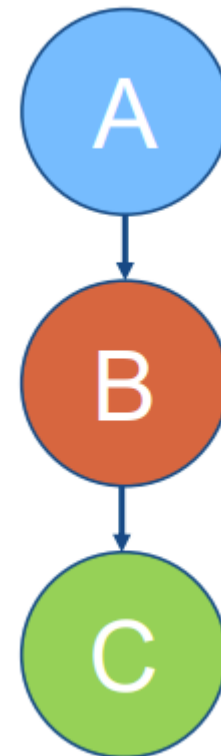
```
    // some data initialization
    q.submit([&](handler &cgh) {
        cgh.parallel_for(1024, [=](auto i) {
            C[i] = A[i] + B[i];
        });
    });
```

```
    q.wait();
    for (int i = 0; i < 1024; i++) {
        std::cout << C[i] << std::endl;
    }
    free(A, q);
    free(B, q);
    free(C, q); }
```

Демонстрация от Intel

USM - зависимость данных в очередях

```
queue q;
int* data = malloc_shared<int>(N, q);
for(int i=0; i<N; i++) data[i] = 10;
q.submit([& (handler &h) {
    h.parallel_for<class taskA>(range<1>(N), [=] (id<1> i) {
        data[i] += 2;
    });
}).wait();
q.submit([& (handler &h) {
    h.parallel_for<class taskB>(range<1>(N), [=] (id<1> i) {
        data[i] += 3;
    });
}).wait();
q.submit([& (handler &h) {
    h.parallel_for<class taskC>(range<1>(N), [=] (id<1> i) {
        data[i] += 5;
    });
}).wait();
for(int i=0; i<N; i++) std::cout << data[i] << " ";
free(data, q);
```



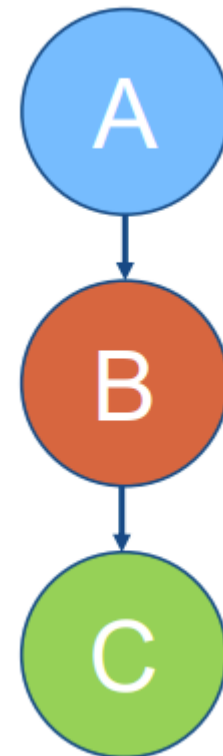
Явный **wait ()**, используемый для обеспечения зависимости данных
wait () блокирует выполнение на хосте

USM - зависимость данных в очередях

```
queue q;
int* data = malloc_shared<int>(N, q);
for(int i=0;i<N;i++) data[i] = 10;
auto e1 = q.submit([& (handler &h){
    h.parallel_for<class taskA>(range<1>(N), [=] (id<1> i) {
        data[i] += 2;
    });
});
auto e2 = q.submit([& (handler &h){
    h.depends_on(e1);
    h.parallel_for<class taskB>(range<1>(N), [=] (id<1> i) {
        data[i] += 3;
    });
});

// non-blocking; execution of host code is possible

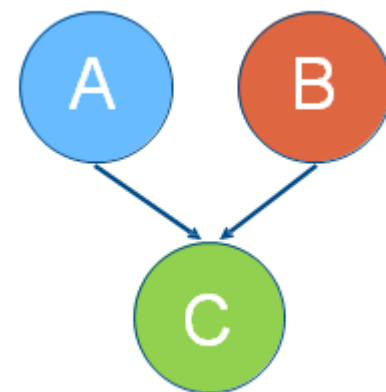
q.submit([& (handler &h){
    h.depends_on(e2);
    h.parallel_for<class taskC>(range<1>(N), [=] (id<1> i) {
        data[i] += 5;
    });
}).wait();
for(int i=0;i<N;i++) std::cout << data[i] << " ";
free(data, q);
```



Используйте метод `depends_on()`, чтобы обработчик группы команд знал, что указанное событие должно быть завершено до того, как указанная задача может быть выполнена.

USM - зависимость данных в очередях

```
queue q;
int* data1 = malloc_shared<int>(N, q);
int* data2 = malloc_shared<int>(N, q);
for(int i=0; i<N; i++) {data1[i] = 10; data2[i] = 10;}
auto e1 = q.submit([& (handler &h) {
    h.parallel_for<class taskA>(range<1>(N), [=] (id<1> i) {
        data1[i] += 2;
    });
});
auto e2 = q.submit([& (handler &h) {
    h.parallel_for<class taskB>(range<1>(N), [=] (id<1> i) {
        data2[i] += 3;
    });
});
q.submit([& (handler &h) {
    h.depends_on({e1, e2});
    h.parallel_for<class taskC>(range<1>(N), [=] (id<1> i) {
        data1[i] += data2[i];
    });
}).wait();
for(int i=0; i<N; i++) std::cout << data[i] << " ";
free(data1, q); free(data2, q);
```



Используйте метод `depends_on()`, чтобы обработчик группы команд знал, что указанное событие должно быть завершено до того, как указанная задача может быть выполнена.

USM - зависимость данных в очередях

```
queue q;  
int* data1 = malloc_shared<int>(N, q);  
int* data2 = malloc_shared<int>(N, q);  
for(int i=0;i<N;i++) {data1[i] = 10; data2[i] = 10;}  
auto e1 = q.parallel_for <class taskA>(range<1>(N), [=](id<1> i) {  
    data1[i] += 2;  
});  
auto e2 = q.parallel_for <class taskB>(range<1>(N), [=](id<1> i) {  
    data2[i] += 3;  
});  
q.parallel_for <class taskC>(range<1>(N), {e1, e2}, [=](id<1> i) {  
    data1[i] += data2[i];  
}).wait();  
  
for(int i=0;i<N;i++) std::cout << data[i] << " ";  
free(data1, q); free(data2, q);
```

Более упрощенный способ указания зависимости в качестве параметра parallel_for

